MAIE 5532: Machine Learning Systems

Week 1: Machine Learning Frameworks and Abstractions

Program: MSc in AI & Entrepreneurship

Term: Fall 2026/27

Instructor: Lefteris NTAFLOS, PhD

The Hong Kong University of Science and Technology

Welcome to Machine Learning Systems

Course Overview:

- **Exploration** of principles, methodologies, and technologies that form the backbone of machine learning systems
- **Balanced emphasis** on theoretical foundations and practical implementations
- **Design principles** and future challenges in ML systems
- **Next-generation applications** and hardware platforms

Quick quiz

Which matters more in industry deployment:

1) Model accuracy?

or

2) System efficiency?

Why?

What matters more in industry deployment?

Take 1 – Model accuracy matters more

- Goal: maximize accuracy
- Use cases: medical imaging, fraud detection, legal NLP
- Pros: fewer errors, higher trust, safer results
- Best for: safety-critical, high-stakes tasks

Take 2 – System efficiency matters more

- Goal: speed, cost, reliability
- Use cases: social feeds, chat assistants, search, recommendations
- Pros: real-time, scalable, cheaper ops
- Best for: high-volume, budget-conscious apps

Use-case dictates emphasis; tiered approach: fast lightweight models for common cases, slower accurate models for edge cases

Course Learning Outcomes

By the end of this course, you will be able to:

1. **Develop a deep understanding** of machine learning systems
2. **Design scalable and efficient systems** for training and deploying ML models
3. **Implement ML systems** using appropriate programming languages and frameworks
4. **Optimize computational resources and algorithms** to improve ML system performance
5. **Evaluate effectiveness and efficiency** of ML systems using relevant metrics and benchmarks

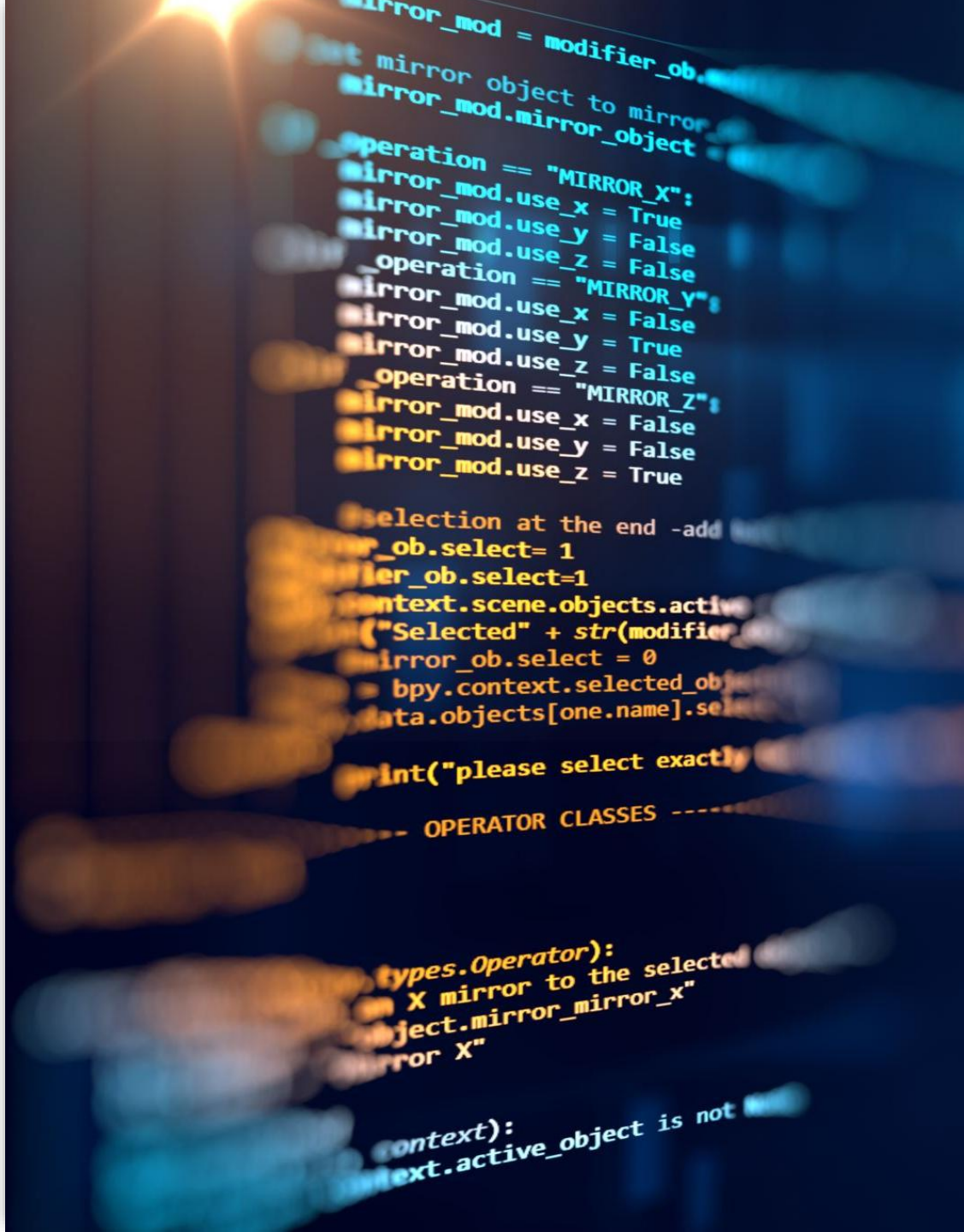
Course Structure - 13 Weeks (2h and 50min / session)

Week	Topic
1	Machine Learning Frameworks and Abstractions
2	Automatic Differentiation
3	Graph-level Optimizations
4	Parallelism
5	Distributed Training
6	Hardware Backend Specialization and Optimizations
7	Machine Learning for Systems
8	Automating ML Compilation
9	Pruning and Sparsity
10	Quantization
11	Neural Architecture Search
12	Knowledge Distillation
13	Federated Learning

Assessment Overview

Course Assessment:

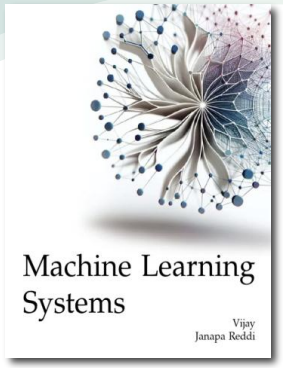
- **Assignments (40%):** 4 programming assignments focusing on practical implementation
- **Midterm Exam (30%):** Theoretical foundations and system design principles
- **Final Project (30%):** System design and implementation

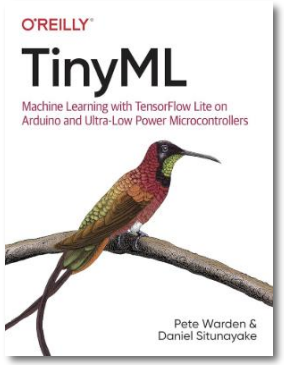


Essential References

Primary Textbooks

- **Book 1:** Vijay Janapa Reddi, "**Machine Learning Systems**", MIT Press, 2026.
  - Volume 1: Introduction to Machine Learning Systems
  - Volume 2: Machine Learning Systems at Scale
- **Book 2:** Pete Warden and Daniel Situnayake, "**TinyML**", O'Reilly Media, Inc., 2019
  - Find it in "O'Reilly for Higher Education", in HKUST Library. Instructions here: https://libguides.hkust.edu.hk/ebook/safari





Today's Required Readings:

- **Book 1 - Volume 1:** ML Frameworks
- **Book 1 - Volume 1**: Neural Computation (especially automatic differentiation and backpropagation background)
- **Book 1 - Volume 1**: Hardware Acceleration
- **Book 1 - Volume 1**: Model Serving
- **Book 2:** Chapter 13: TensorFlow Lite for Microcontrollers
- **Book 2:** Chapter 1: Introduction (TinyML context)

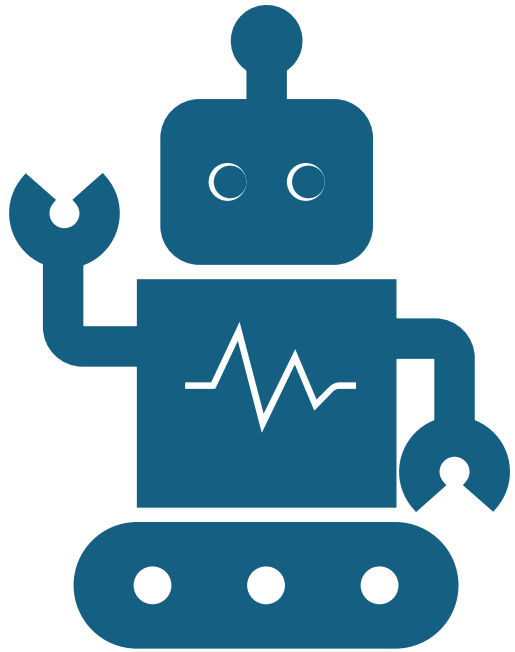
Why Machine Learning Systems Matter

The Revolution is Here

- Machine learning has **revolutionized problem-solving** across various fields
- **Computer vision, natural language processing, robotics** - all transformed
- **Advanced ML systems** are the driving force behind this success

The Challenge

- Models are becoming **increasingly large and computationally intensive**
- Need for **efficient training and deployment** of complex models
- **Multiple devices and distributed resources** are now the norm



Quick quiz

Which app class is most systems-bound:

1) CV (Computer Vision),

2) NLP (Natural Language Processing),

or

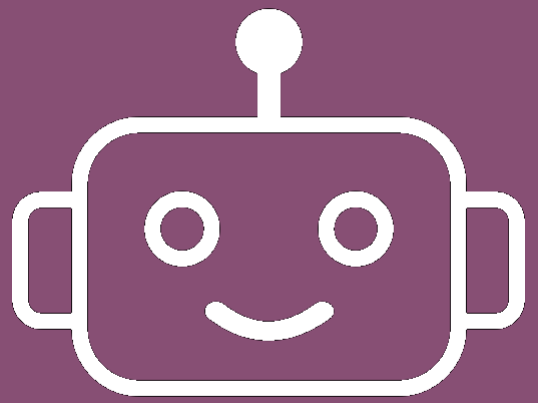
3) Robotics?”

Being “systems-bound” means the limiting factor is the whole system’s engineering and constraints, not just the machine learning model.

Which app class is most systems-bound:

Most systems-bound tends to be robotics.

Why: robotics blends perception (CV/NLP), real-time control, and stringent latency, power, and reliability constraints on embedded hardware. The loop from sensing to actuation amplifies physics-based deadlines and energy use, making system-level bottlenecks (compute, memory, communication, power) dominant.



Machine Learning Frameworks and Abstractions

Learning Objectives

- **Trace the evolution** of ML frameworks from early numerical libraries to modern deep learning systems
- **Analyze framework fundamentals** including tensor data structures, computational graphs, execution models
- **Differentiate between ML framework architectures**, execution strategies, and development tools
- **Compare framework specializations** across cloud, edge, mobile, and TinyML applications

What is a Machine Learning Framework?

Definition

A Machine Learning Framework is a software platform that provides tools and abstractions for designing, training, and deploying machine learning models.

Key Functions:

Bridges user applications with infrastructure

Enables algorithmic expressiveness through computational graphs and operators

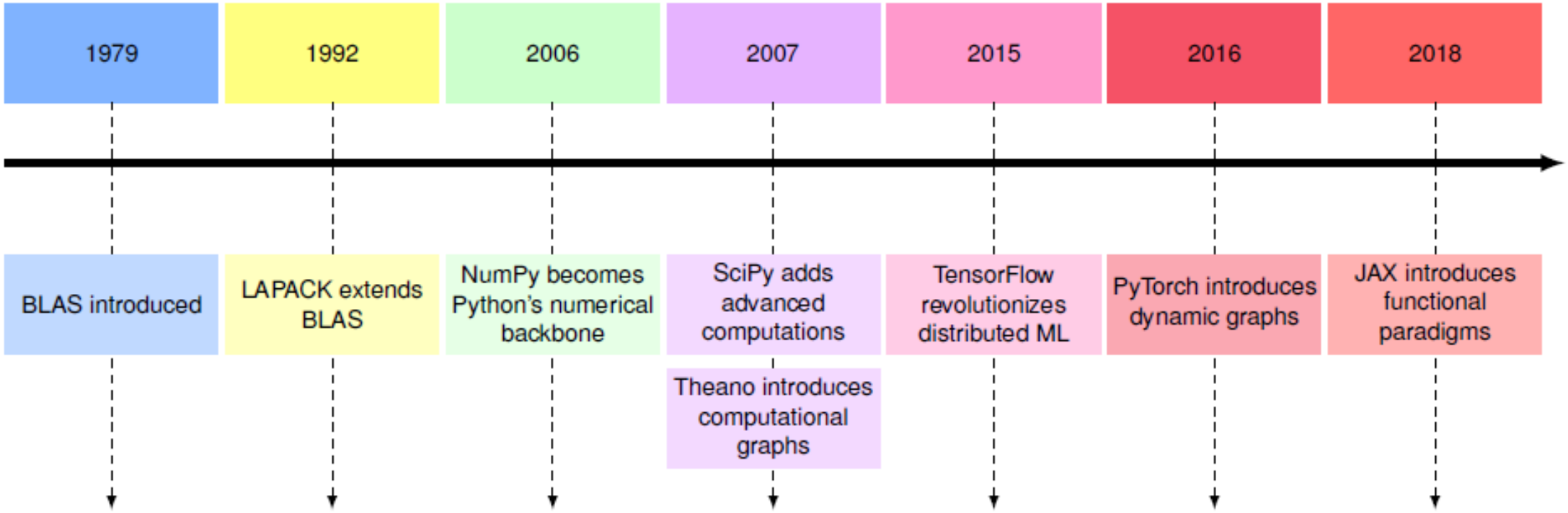
Workflow orchestration across the ML lifecycle

Hardware optimization with schedulers and compilers

Scalability for distributed and edge systems

Extensibility to support diverse use cases

Evolution of Machine Learning Frameworks



Pattern: Each layer builds upon robust foundations of predecessors

Source/adapted from: MLSysBook Vol. I, ML Frameworks

Early Foundations: BLAS and LAPACK

BLAS (Basic Linear Algebra Subprograms) - 1979

- **Essential matrix operations** - the computational backbone of ML
- **Matrix-matrix and matrix-vector multiplications**
- Foundation for all modern ML computations

$$\begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

LAPACK (Linear Algebra Package) - 1992

- **Extended BLAS capabilities**
- **Matrix decompositions, eigenvalue problems, linear system solutions**
- **Layered approach** - building complex operations from fundamental matrix computations

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

Impact: Established the pattern of progressive abstraction in ML frameworks

The Python Revolution: NumPy and SciPy

NumPy (2006) – Numerical Python

- **n-dimensional array objects** and essential mathematical functions
- **Efficient interface** to underlying BLAS and LAPACK operations
- **High-level array operations** while maintaining performance
- Became the **fundamental package** for numerical computation in Python

SciPy (2001) – Scientific Python

- **Built on top of NumPy**
- **(2007) Specialized functions** for optimization, linear algebra, signal processing
- **Further abstraction** from basic matrix operations to sophisticated numerical computations

Pattern: Layered architecture starting from fundamental matrix operations

First-Generation ML Frameworks

Weka (1993) - University of Waikato

- **One of the earliest ML frameworks**
- **Abstracted matrix operations** into data mining tasks
- **Limited by Java implementation** and focus on smaller-scale computations

Scikit-learn (2007)

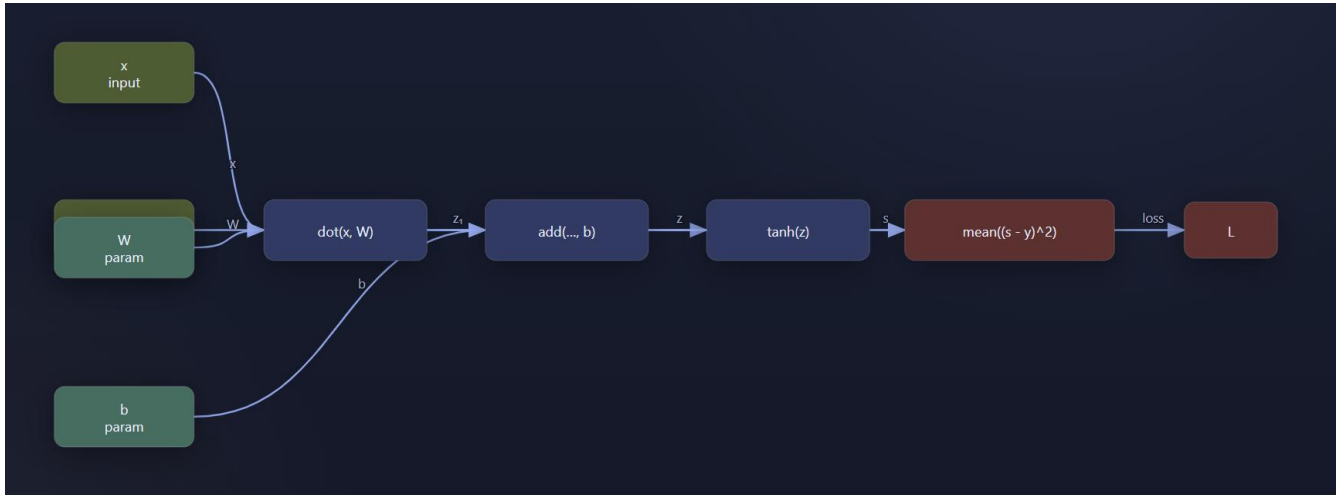
- **Built upon NumPy and SciPy foundation**
- **Transformed basic matrix operations** into intuitive ML algorithms
- **Simple fit() method calls** hiding complex matrix computations
- **Clean APIs** became a defining characteristic of modern ML frameworks

Theano - The Game Changer

Theano: The Revolutionary Framework (2007)

Developed at MILA (Montreal Institute for Learning Algorithms)

theano



Two Revolutionary Concepts:

1. **Computational Graphs**
   - Mathematical operations as directed graphs
   - Matrix operations as nodes, data flowing between them
   - Enabled automatic differentiation and optimization
2. **GPU Acceleration**
   - Automatically route operations to GPU hardware
   - Dramatically accelerated matrix computations
   - Made deep learning practical

Impact: Established design patterns that influenced PyTorch and others

Image source: https://computer.howstuffworks.com/

The Deep Learning Revolution

Deep learning: A subset of machine learning that uses multi-layer neural networks to learn hierarchical representations from data

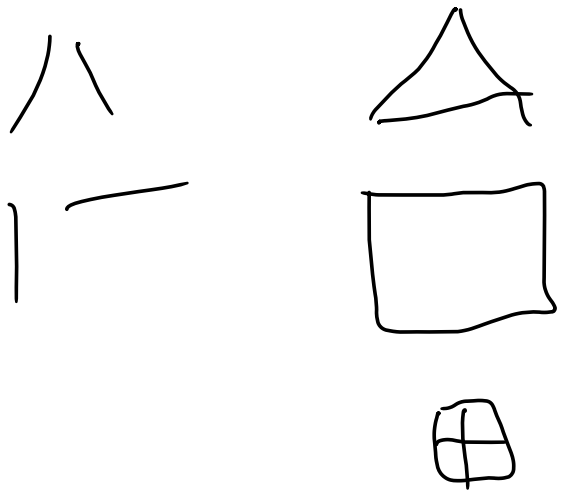
Challenges Faced:

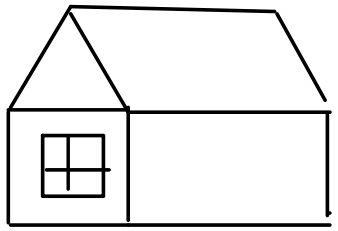
- **Massive scale of computations** - billions of matrix operations
- **Complex gradient calculations** through deep networks
- **Need for distributed processing**

Deep Learning: The Big Idea

- Learns layers of neurons
- Early: lines/curves
- Later: parts → concepts

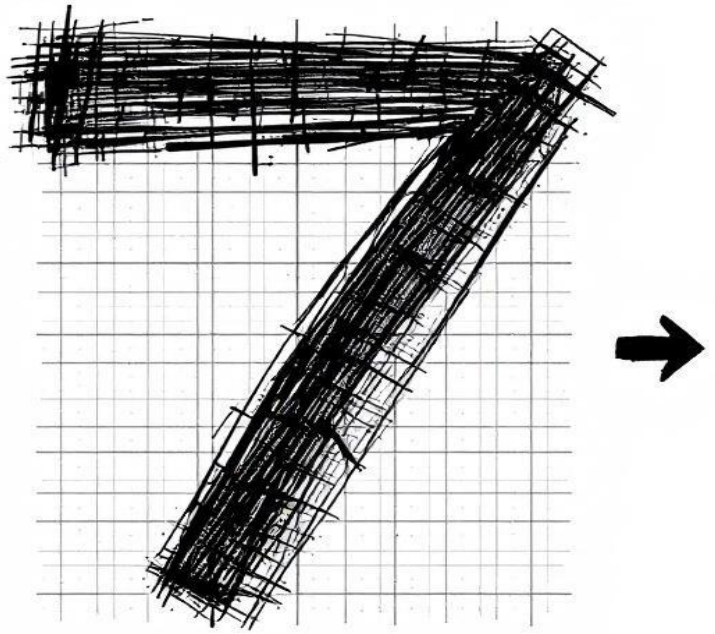
Neurons -> units of computation

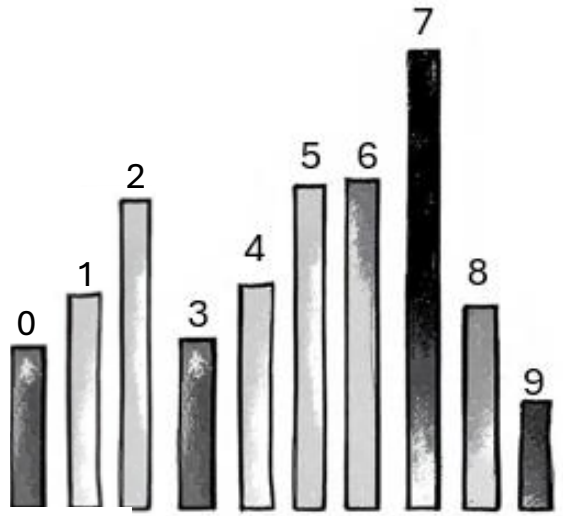




Deep Learning: Example (Digits)

- Task: recognize 0–9
- Input: 28×28 image
- Output: probabilities

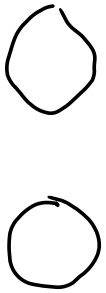


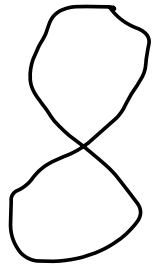


Deep Learning: Example (Digits)

Neurons -> units of computation

What Each Layer (set of neurons) Does

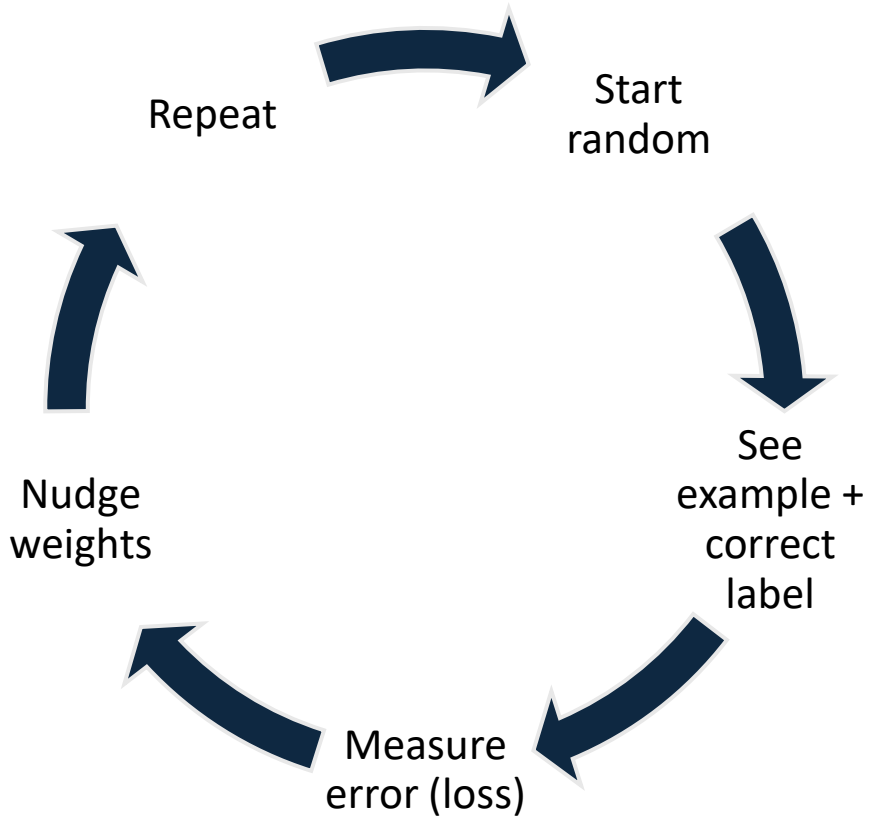




Layer 1: strokes/edges **Layer 2: loops/corners** **Layer 3+: digit shape**

Deep Learning: How Learning Happens

What Each Layer Does



It's practice with feedback, thousands of times.

Deep Learning: Gradients Intuition

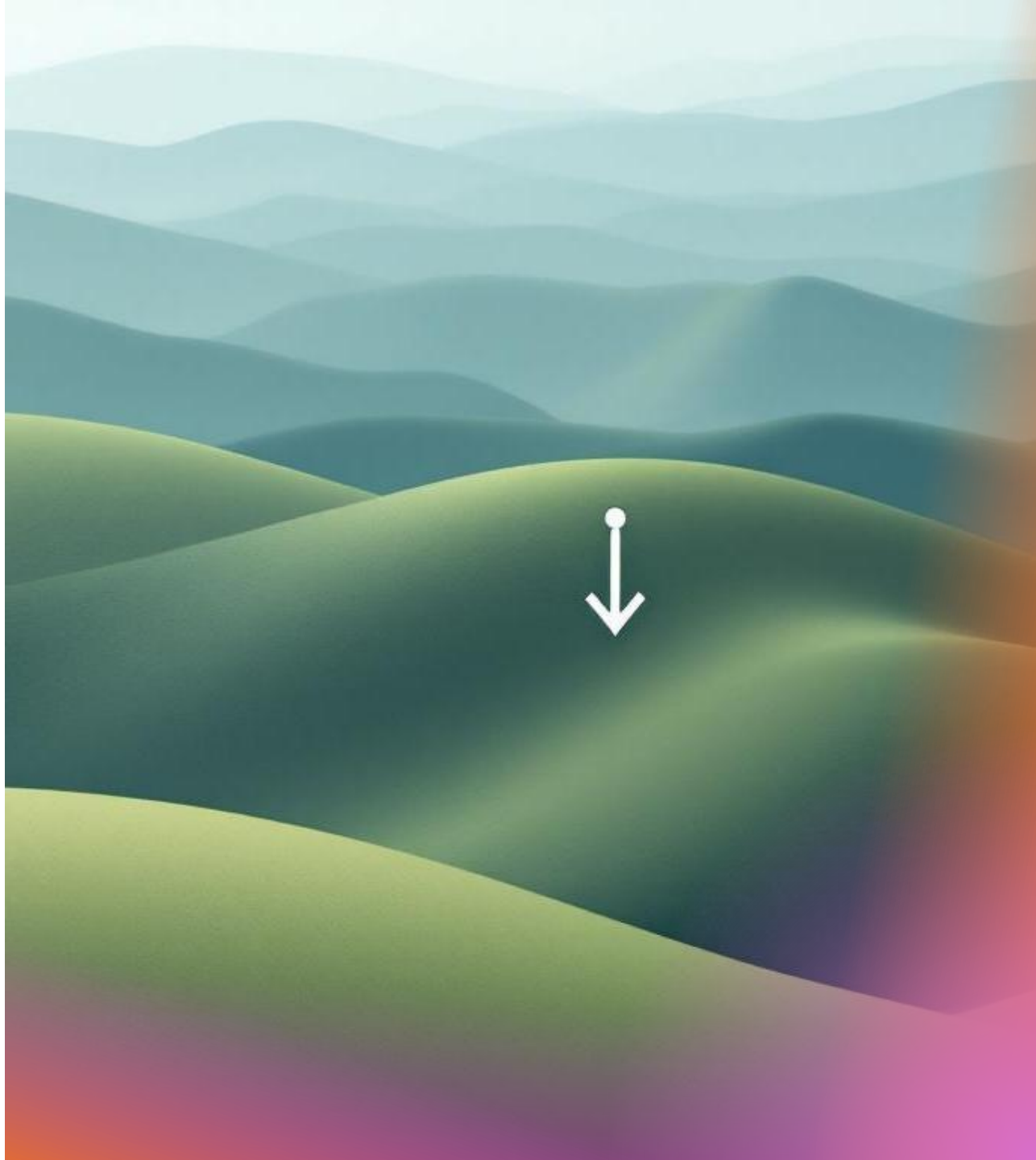
Walk downhill in fog

Gradient = steepest descent

The gradient tells us how to tweak each parameter to reduce error fastest.

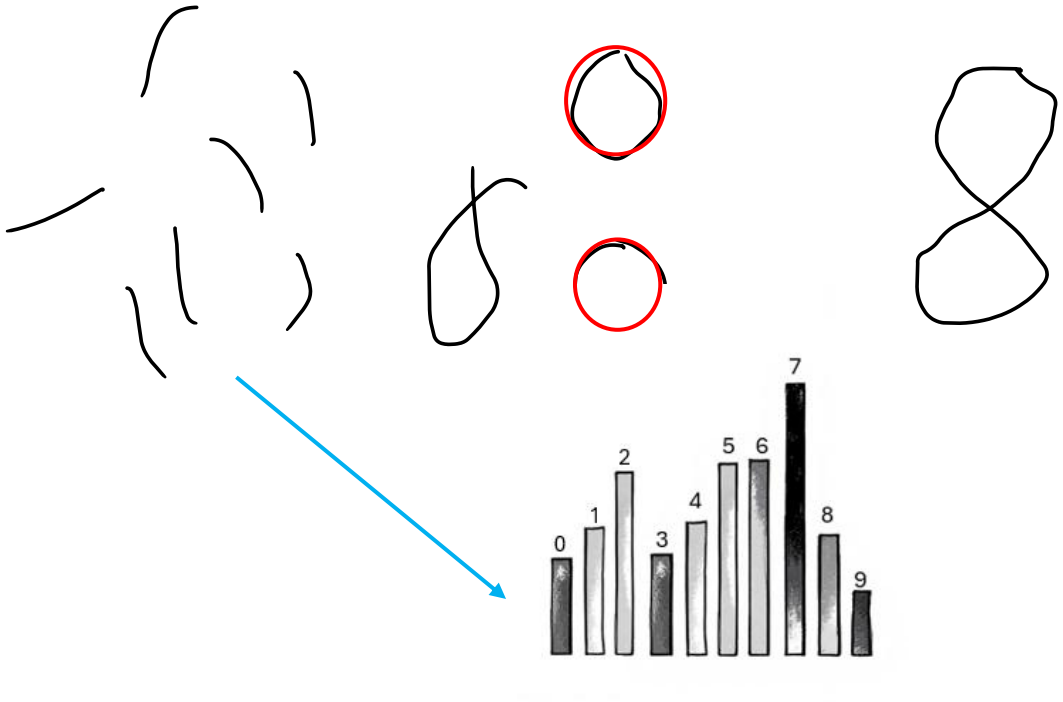
How we calculate gradients?

We use **Automatic Differentiation (AD)**, a technique for computing exact derivatives of functions implemented as programs, by applying the chain rule to the sequence of elementary operations.



Deep Learning: Backpropagation

- **Forward pass**: make a guess
- **Backward pass**: assign responsibility
- Update each layer



Forward pass

- $z = w_1x_1 + w_2x_2 + b$ (weighted sum)
- $a = \sigma(z)$ (activation function e.g. sigmoid)
- $L = \frac{1}{2}(y - a)^2$ (loss – squared error)
- w: weights, b: bias, x: input, y: output

Backward pass (reverse mode AD)

- $\partial L/\partial a = -(y - a)$
- $\partial a/\partial z = \sigma(z)(1 - \sigma(z))$
- $\partial L/\partial z = (\partial L/\partial a)(\partial a/\partial z)$
- $\partial L/\partial w_1 = x_1\, \partial L/\partial z$, $\partial L/\partial w_2 = x_2\, \partial L/\partial z$
- $\partial L/\partial b = \partial L/\partial z$

Quiz

1) Which layer in the software stack most directly accelerates matrix multiplications used by ML frameworks?
A. Scikit-learn
B. BLAS/cuBLAS
C. GUI toolkits
D. Web servers

2) Which statement best captures the role of an ML framework?
A. Replaces the need for linear algebra libraries
B. Provides abstractions (tensors/graphs) and maps user code to optimized kernels and hardware
C. Only offers data visualization tools
D. Eliminates the need for training data

3) What was a key contribution of Theano to modern DL frameworks?
A. Distributed parameter servers
B. Static web dashboards
C. Computational graphs with automatic differentiation and GPU mapping
D. Rule-based feature engineering

4) In deep learning, what do earlier versus later layers typically learn?
A. Early: concepts; Late: edges
B. Early: edges/strokes; Late: parts and full concepts
C. Both learn identical features
D. Early: labels; Late: data loading

5) Why is reverse-mode automatic differentiation (backprop) preferred for neural networks?
A. It approximates gradients randomly
B. It computes gradients of all parameters with cost similar to one forward pass
C. It only works for linear models
D. It avoids the chain rule

Quiz - Answers

1. B. BLAS/cuBLAS

- Explanation: BLAS (and vendor variants like cuBLAS) provide highly optimized matrix/vector kernels that frameworks call under the hood to accelerate GEMM and related ops.

2. B. Provides abstractions (tensors/graphs) and maps user code to optimized kernels and hardware

- Explanation: Frameworks bridge user programs to infrastructure, orchestrate the ML lifecycle, and dispatch to compilers/kernels across CPUs, GPUs, and accelerators.

3. C. Computational graphs with automatic differentiation and GPU mapping

- Explanation: Theano pioneered symbolic graphs enabling AD and optimization, and it could route graph segments to GPUs—designs later refined by TF, PyTorch, and JAX.

4. B. Early: edges/strokes; Late: parts and full concepts

- Explanation: Hierarchical feature learning: simple patterns in early layers, combined into parts and then task-level concepts in deeper layers.

5. B. It computes gradients of all parameters with cost similar to one forward pass

- Explanation: Reverse-mode AD propagates gradients from outputs to inputs efficiently, making it scalable for models with many parameters like neural nets.

Deep Learning: Heavy computation (matrices)

- Billions of multiplications
- Many passes (epochs)
- Millions–billions of weights

Deep Learning: Sense of Scale

- 5M weights → ~billions ops/update
- 50k updates → trillions ops
- Use GPUs/TPUs

A Graphics Processing Unit (GPU) is a specialized electronic circuit that accelerates the creation of images and visual data for display on a screen by performing many calculations simultaneously. GPUs are essential for tasks requiring intense parallel processing, such as rendering 3D graphics, video editing, machine learning, and artificial intelligence.

Tensor Processing Unit (TPU)

A custom hardware accelerator developed by Google, designed specifically for accelerating AI and machine learning computations.

A tensor is a multidimensional array of numbers that is a generalization of scalars and vectors, with scalars being rank zero tensors, vectors being rank one tensors, matrices being rank two tensors and so on so forth.

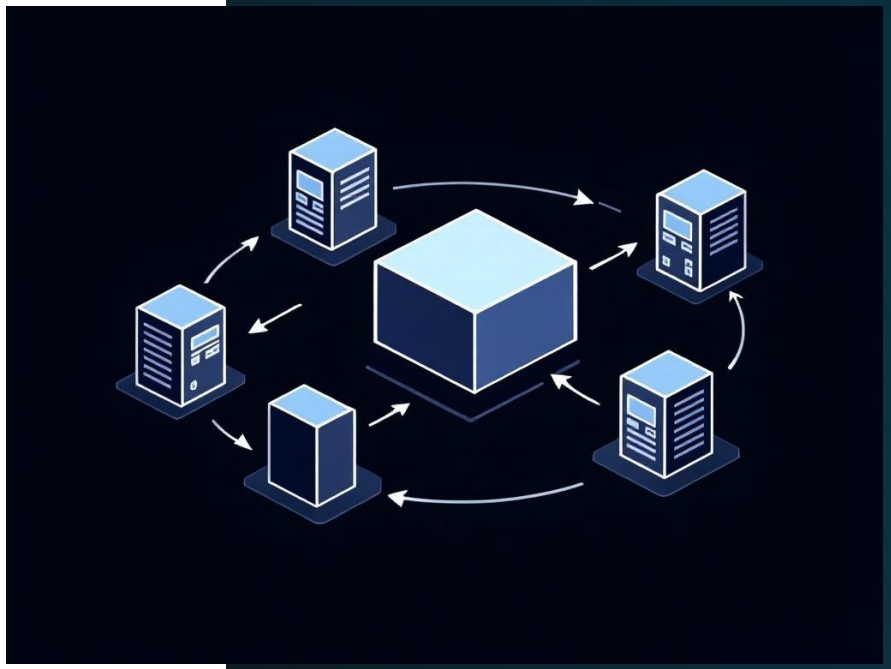
Deep Learning: Distributed training (data parallel)

Data parallel: split data, average gradients

- Identical model on multiple devices;
- Each processes a different data subset
- Compute gradients locally on each device; then average/aggregate gradients to update the shared model

Model parallel:

- split the model across devices;
- forward and backward passes pass activations/gradients through the partitioned layers



The Deep Learning Revolution

Key Developments:

Caffe (2013) - UC Berkeley

- **Specialized convolutional operation implementations**
- **Optimized matrix patterns** for computer vision tasks

Torch (2002) - NYU

- **Immediate execution** of operations (eager execution)
- **Flexible interface** for neural network implementations
- **Balanced abstractions with efficient low-level operations**

TensorFlow: The Distributed Computing Revolution (2015)

Developed by Google Brain Team

Revolutionary Approach:

- **Treated matrix operations** as distributed computing problem
- **Static computational graphs** split across multiple devices
- **Enabled training of unprecedented model sizes**

Key Features:

- **Aggressive optimization** through kernel fusion
- **Memory planning** and pre-allocation
- **Cross-platform deployment** capabilities
- **Production-focused** design philosophy
- **Impact: Made large-scale distributed training practical**

Kernel fusion is an optimization technique used in GPU and CPU compute to improve performance by combining multiple small operations (kernels or functions) into a single, larger kernel. This reduces overhead and can improve memory bandwidth utilization.

PyTorch: The Dynamic Revolution (2016)

Developed by Facebook AI Research

Revolutionary Approach:

- **Dynamic computational graphs** modified on-the-fly
- **"Define-by-run"** execution model
- **Immediate feedback** during development

Key Advantages:

- **Easy debugging** and understanding
- **Natural integration** with Python debugging tools
- **Flexible model development** for research
- **Intuitive programming** model
- **Impact: Became the preferred choice for ML research community**

JAX: Functional Transformations for High-Performance ML (2018) - developed by Google

- Transform numerical Python/NumPy functions
- Functional programming paradigm; immutable arrays
- XLA (accelerated linear algebra)-backed just-in-time compilation for CPUs/GPUs/TPUs

Why It Matters:

- Research velocity with concise, pure functions
- Performance: kernel fusion, layout optimization, device placement via XLA
- Scales from single device to multi-accelerator setups (TPUs/GPUs)

Just-in-Time (JIT) compilation is a method for executing computer code that translates it into native machine code at runtime. It is a hybrid approach that combines the best aspects of traditional interpreters (flexibility) and static, ahead-of-time (AOT) compilers (performance).

XLA, which stands for Accelerated Linear Algebra, is an open-source compiler infrastructure designed to improve the performance of machine learning models and other numerical computations, particularly those involving large-scale array operations.

Framework Comparison Table

	TensorFlow	PyTorch	JAX
Graph Type	Static (1.x), Dynamic (2.x)	Dynamic	Functional transformations
Programming Model	Imperative (2.x), Symbolic (1.x)	Imperative	Functional
Core Data Structure	Tensor (mutable)	Tensor (mutable)	Array (immutable)
Execution Mode	Eager (2.x default), Graph	Eager	Just-in-time compilation
Automatic Differentiation	Reverse mode	Reverse mode	Forward and Reverse mode
Hardware Acceleration	CPU, GPU, TPU	CPU, GPU	CPU, GPU, TPU

Eager execution in TensorFlow is an imperative programming environment where operations are evaluated immediately as they are called, rather than building a computational graph to be run later. This contrasts with TensorFlow's traditional graph mode, where operations are first added to a symbolic graph and then executed in a TensorFlow session.

Hardware Impact on Framework Design

GPU Revolution (NVIDIA CUDA - 2007)

- **General-purpose computing on GPUs**
- **Orders of magnitude speedup** for matrix operations
- **Parallel processing** of thousands of elements simultaneously
- **Specialized Accelerators**

Google TPUs (2016) - purpose-built for tensor operations

- **Systolic array architectures** - efficient for matrix multiplication
- **Mobile accelerators** - Apple Neural Engine, Qualcomm NPUs

Impact on Frameworks:

- **Specialized compilation strategies**
- **New optimization opportunities**
- **Hardware-specific operator implementations**

Google TPU (Image source: https://www.servethehome.com)

Quiz

1) Why do deep learning workloads map well to GPUs?
A. GPUs have larger caches than CPUs
B. GPUs excel at massively parallel dense linear algebra like GEMMs/convolutions
C. GPUs run higher clock speeds than CPUs
D. GPUs avoid using memory entirely

2) In data-parallel training, what communication step typically synchronizes model replicas each iteration?
A. Parameter server RPCs
B. All-to-all exchange of activations
C. All-reduce of gradients
D. Broadcasting the dataset

3) Which statement best describes JAX's programming model?
A. Imperative mutable tensors with static graphs
B. Pure functions over immutable arrays transformed by jit/grad/vmap/pjit
C. Graphs defined in a separate configuration language
D. Only forward-mode differentiation is supported

4) What benefit does kernel fusion provide on accelerators?
A. Increases model accuracy
B. Reduces op launch overhead and memory traffic by combining ops
C. Eliminates the need for backpropagation
D. Doubles device memory capacity

5) When scaling batch size across more GPUs, which training adjustment is commonly recommended?
A. Decrease learning rate proportionally to batch size
B. Keep learning rate fixed regardless of batch
C. Use the linear scaling rule and warmup the learning rate
D. Disable mixed precision to stabilize training

Quiz - Answers

1) B. GPUs excel at massively parallel dense linear algebra like GEMMs/convolutions

Explanation: DL layers are dominated by matrix multiplies and convs, which map to thousands of parallel threads/cores and high memory bandwidth on GPUs.

2) C. All-reduce of gradients

Explanation: Each replica computes local gradients; an all-reduce averages/sums them so every replica applies the same update and stays synchronized.

3) B. Pure functions over immutable arrays transformed by jit/grad/vmap/pjit

Explanation: JAX is functional-first; transformations handle compilation, autodiff, batching, and parallelism while keeping code NumPy-like.

4) B. Reduces op launch overhead and memory traffic by combining ops

Explanation: Fusion merges sequences of operations into a single kernel, improving arithmetic intensity and reducing reads/writes to memory.

5) C. Use the linear scaling rule and warmup the learning rate

Explanation: As global batch increases, scale LR roughly linearly and add a warmup phase to avoid optimization instability early in training. Mixed precision typically helps, not hurts, stability with proper loss scaling.

Framework Fundamentals: Four Key Layers

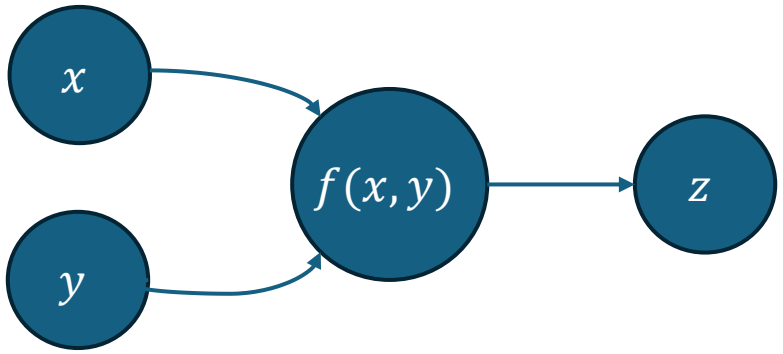
1. Fundamentals Layer	2. Data Handling Layer	3. Developer Interface Layer	4. Execution and Abstraction Layer
• **Computational graphs** - structural basis using DAGs • Enable automatic differentiation and optimization	• **Specialized data structures** (tensors) • **Memory management** and device placement	• **Programming models** (imperative vs symbolic) • **Execution models** (eager vs graph vs JIT)	• **Core operations** optimized for diverse hardware • **Resource allocation** and memory management

Computational Graphs - The Foundation

What are Computational Graphs?

- **Directed Acyclic Graphs (DAGs)** representing ML models
- **Nodes represent operations** (matrix multiplication, convolution)
- **Edges represent data flow** between operations

Example: Simple Computation: $z = x \times y$



System Benefits:

- **Automatic differentiation** capabilities
- **Operation scheduling** and optimization
- **Distributed execution** across devices
- **Memory planning** and resource allocation

Static vs Dynamic Computational Graphs

Static Graphs (TensorFlow 1.x style)

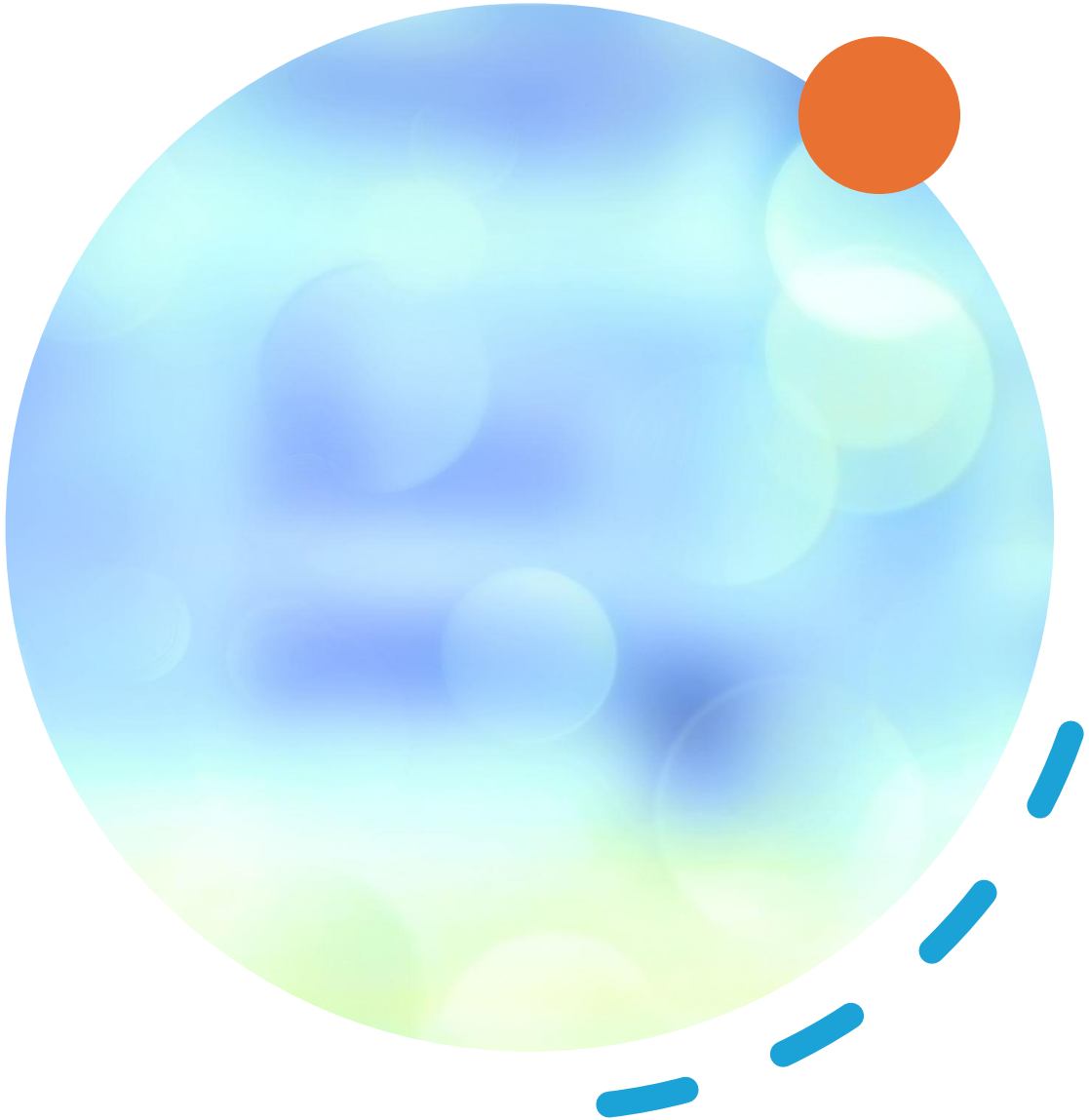
- **"Define-then-run"** execution model
- **Complete graph specification** before execution
- **Global optimization** opportunities
- **Efficient production deployment**

Dynamic Graphs (PyTorch style)

- **"Define-by-run"** execution model
- **Graph construction during execution**
- **Immediate feedback** and easy debugging
- **Flexible control flow**

Trade-offs:

- **Static**: Better optimization vs. **Dynamic**: Better flexibility
- **Static**: Production efficiency vs. **Dynamic**: Development ease



Memory Management: Static vs Dynamic

Static Graphs

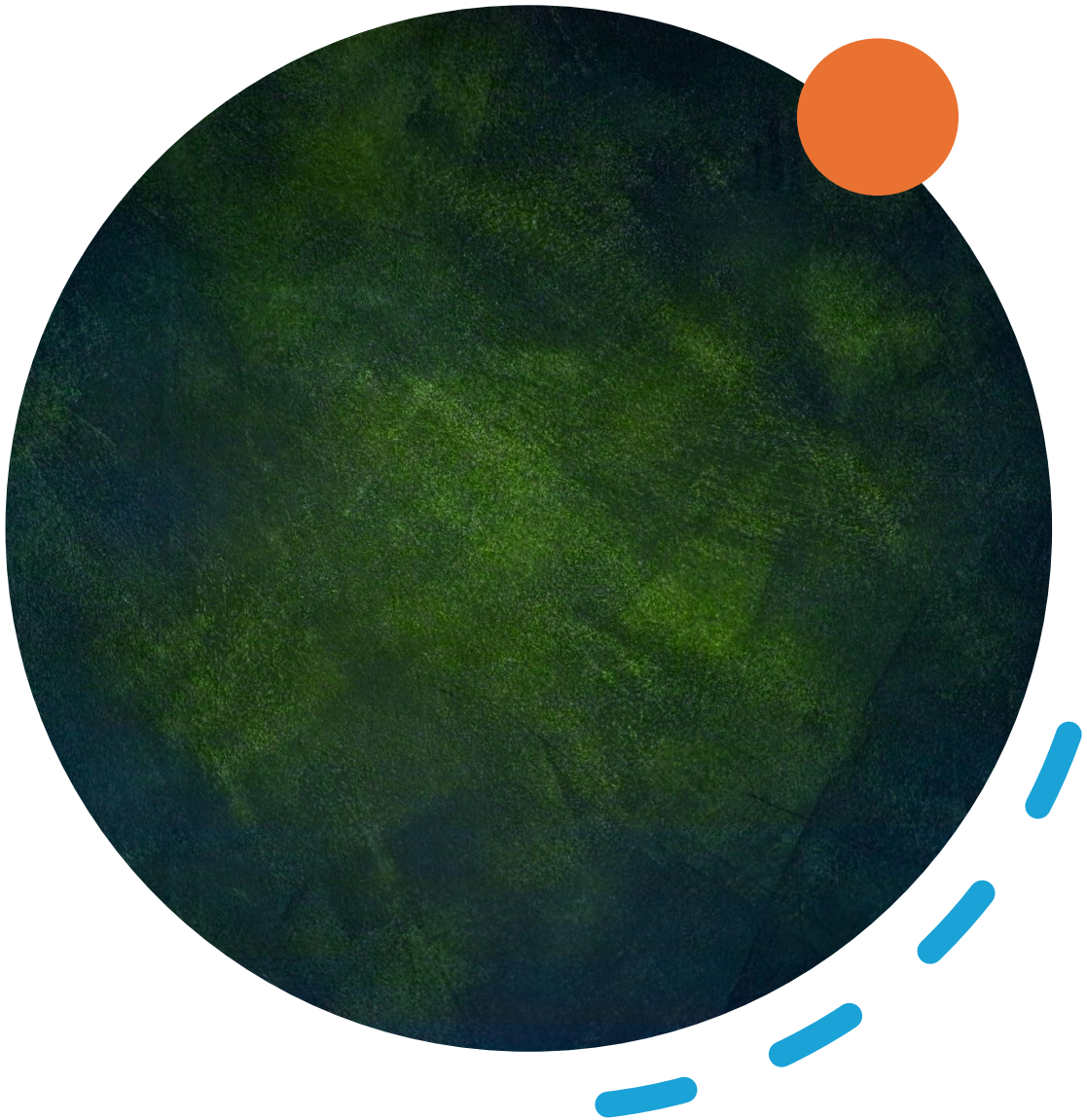
- **Precise memory planning** before execution
- **Optimized allocation** strategies
- **Memory reuse** opportunities
- **Lower memory overhead**

Dynamic Graphs

- **Runtime memory allocation**
- **Flexible but potentially less efficient**
- **Memory fragmentation** risks
- **Higher memory overhead**

System Consequences:

- **Device placement** optimization
- **Memory bandwidth** utilization
- **Resource constraint** handling



Automatic Differentiation: The Engine of Learning

What is Automatic Differentiation?

Systematic computation of derivatives through complex operations

Enables gradient-based optimization for neural network training

Chain rule application across computational graphs

Example: $f(x) = x^2 \times sin(x)$

AD decomposes into:

1. **a** $= x^2 \rightarrow$ derivative $2x$
2. **b** $= sin(x) \rightarrow$ derivative: $cosx(x)$
3. **Product rule** → final derivative

```
def f(x):
    a = x * x          # Square operation
    b = sin(x)         # Sine operation
    return a * b       # Product operation
```

Forward Mode Automatic Differentiation

Approach: Compute derivatives alongside original computation

- Example with Dual Numbers:

```
x = (2.0, 1.0)      # (value, derivative)
a = (4.0, 4.0)      # x² and derivative 2x
b = (0.909, -0.416) # sin(x) and derivative cos(x)
result = (3.637, 2.805) # Final value and derivative
```

- **Characteristics:**
- Cost scales with input variables
- Constant memory usage
- Efficient for few inputs, many outputs
- Good for sensitivity analysis

Reverse Mode Automatic Differentiation

Approach: Backward pass-through computation graph

Two-Phase Process:

1) Forward Pass: Store intermediate values

```
x = 2.0
a = 4.0       # x * x
b = 0.909     # sin(x)
c = 3.637     # a * b
```

2) Backward Pass: Compute gradients

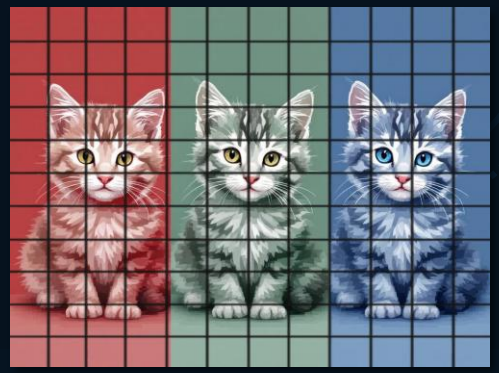
```
dc/dc = 1.0
dc/da = b = 0.909
dc/db = a = 4.0
dc/dx = (2*x*dc/da) + (cos(x)*dc/db) = 2.805
```

Perfect for neural networks: One output (loss), many parameters

Chain rule: c is a variable that is a function of two intermediate variables a and b, which in turn depend on x. To calculate the total derivative dc/dx we must add the individual contributions of:

- dc/dx via a: (∂c/∂a) · (da/dx), and
- dc/dx via b: (∂c/∂b) · (db/dx)

Data Structures: Tensors



What are Tensors?

- **Generalization** of scalars, vectors, and matrices
- **Multi-dimensional arrays** with additional properties
- **Unified representation** for all ML data

Tensor Hierarchy:

- **Rank 0**: Scalar (single value) 1
- **Rank 1**: Vector (1D array) $\begin{bmatrix}1 & 2 & 3\end{bmatrix}$
- **Rank 2**: Matrix (2D array) $\begin{bmatrix}1 & 2\\3 & 4\end{bmatrix}$
- **Rank 3+**: Higher-dimensional arrays $\begin{bmatrix}\begin{bmatrix}1 & 2\\3 & 4\end{bmatrix} & \begin{bmatrix}5 & 6\\7 & 8\end{bmatrix}\\\begin{bmatrix}9 & 10\\11 & 12\end{bmatrix} & \begin{bmatrix}13 & 14\\15 & 16\end{bmatrix}\end{bmatrix}$

Example: Color Image as 3D Tensor

- **Height × Width × Channels** (RGB)
- **Batch processing**: 4D tensor (Batch × Height × Width × Channels)

Tensor Properties and Management

Key Properties:

- **Shape/Dimensionality** - tensor dimensions
- **Data type** - float32, int8, etc.
- **Memory layout** - stride patterns for performance
- **Device placement** - CPU, GPU, TPU location

Memory Layout Challenges:

- **Tensors are multi-dimensional**
- **Physical memory is linear**
- **Stride patterns** map multi-dimensional indices to memory addresses
- **Cache efficiency** depends on memory access patterns

Type Systems:

- **float32** - standard for training
- **int8/int16** - efficient inference
- **Mixed-precision** training approaches

Programming Models: Symbolic vs Imperative

Symbolic Programming

- Abstract representations constructed first
- Delayed evaluation until explicitly executed
- Global optimization opportunities

Example: TensorFlow 1.x style

```
# Define graph
x = tf.placeholder(tf.float32)
y = tf.matmul(x, weights)

# Execute separately
with tf.Session() as sess:
    result = sess.run(y, feed_dict={x: data})
```

Imperative Programming

- Immediate execution as code runs
- Natural debugging experience
- Dynamic control flow

Example: PyTorch style

```
# Immediate execution
x = torch.tensor(data)
y = torch.matmul(x, weights)  # Executes immediately
```

Execution Models

Eager Execution

- **Operations executed immediately** when called
- **Intuitive development** experience
- **Easy debugging** with immediate feedback
- **Natural Python integration**

Graph Execution

- **Pre-defined computation structure**
- **Optimized execution** planning
- **Better production performance**
- **Limited runtime flexibility**

Just-In-Time (JIT) Compilation

- **Compile at runtime** for optimization
- **Balance flexibility and performance**
- **Adapt to actual data types/shapes**
- **Example**: PyTorch JIT, JAX compilation

Framework Specialization Across Environments

Environment	Characteristics	Example Frameworks	Key Optimizations
Cloud	Abundant resources, scalability	TensorFlow, PyTorch	Distributed computing, large models
Edge	Moderate resources, low latency	TensorFlow Lite, Edge Impulse	Real-time inference, heterogeneous hardware
Mobile	Power constraints, varied hardware	TensorFlow Lite, Core ML	Energy efficiency, model compression
TinyML	Severe constraints, microcontrollers	TensorFlow Lite Micro	Extreme compression, static allocation *

* Static memory allocation reserves a fixed amount of memory during compile time, is faster to access, simpler to use, and is typically stored on the stack, though it requires memory needs to be known beforehand. Dynamic memory allocation, conversely, occurs during runtime, offers flexibility to change memory size, uses the heap, but requires careful management to prevent memory leaks and is generally slower.

TinyML: The Extreme Specialization

What is TinyML?

"If you can run a neural network model at an energy cost of below 1 mW, it makes a lot of entirely new applications possible."

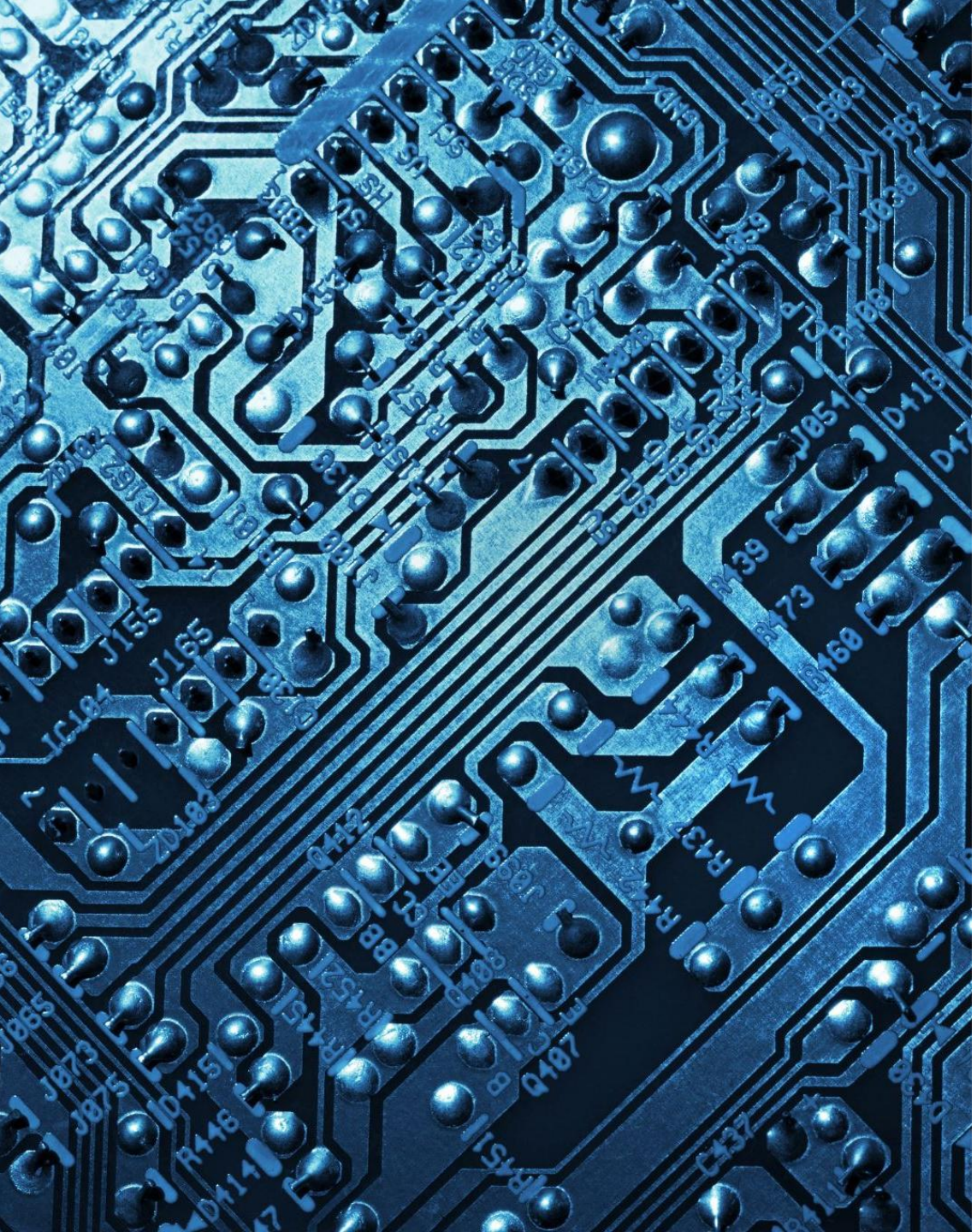
Key Characteristics:

- Energy consumption < 1 mW
- Coin battery lifetime of 1 year
- Fits into any environment
- No human intervention required

Coin cell batteries – Image source: Wikipedia

Example Applications:

- **"Peel-and-stick sensors"** for environmental monitoring
- **Always-on wake word detection** (14KB models on DSPs)
- **Predictive maintenance** sensors
- **Wildlife monitoring** devices



TensorFlow Lite for Microcontrollers

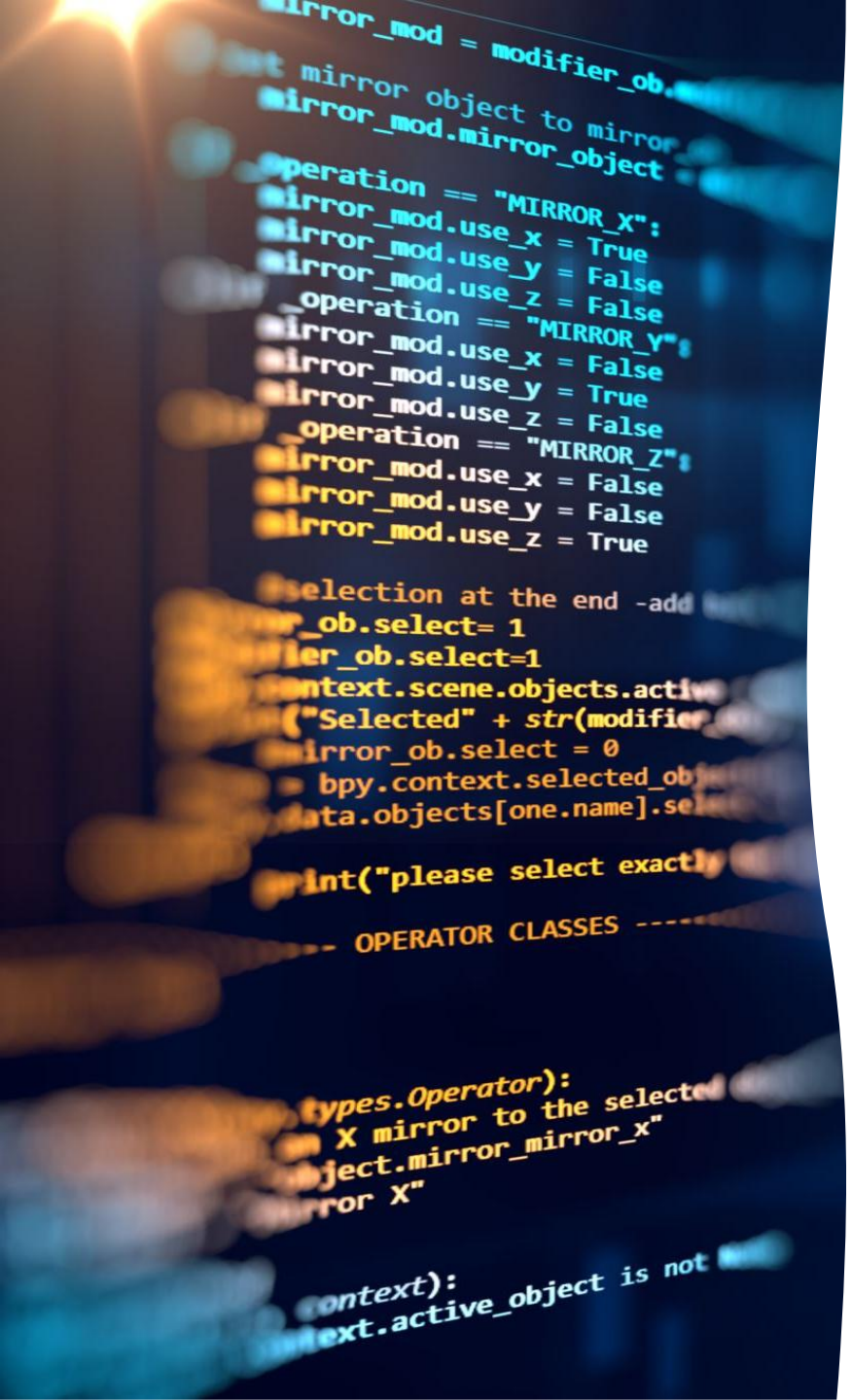
Design Requirements:

- No operating system dependencies
- No standard C/C++ library dependencies at linker time
- No floating-point hardware expected
- No dynamic memory allocation
- Requires C++11 support
- Expects 32-bit processors

TensorFlow Lite

Why These Constraints?

- **Microcontrollers have limited resources** (few KB of memory)
- **Long-running applications** need reliable memory management
- **Binary size is critical** (sprintf() alone can take 20KB)
- **Embedded systems often lack** standard library support



TensorFlow Lite Micro: Architecture Decisions

* During Interpretation, the runtime reads and executes a model **directly from a serialized format** (FlatBuffers) without generating new source code or recompiling the project for every model. Read more on Flatbuffers here.

Interpretation vs Code Generation

Choice: **Model interpretation** rather than code generation

Benefits of Interpretation:

- **Upgradability** - easy framework updates
- **Multiple models** support without code duplication
- **Model replacement** without recompilation
- **Inline data** from FlatBuffers format *

Project Generation Approach:

- **Copy required source files** without modification
- **IDE-specific project files** for easy building
- **External dependencies** included in project
- **Upgradable** through standard merge tools

Building for Different Platforms

Default Makefile Build (Linux):

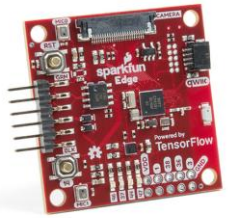
```
make -f tensorflow/lite/micro/tools/make/Makefile test
```

Target-Specific Build (SparkFun Edge):

```
make -f tensorflow/lite/micro/tools/make/Makefile \
TARGET="sparkfun_edge" micro_speech_bin
```

IDE Project Generation (Mbed):

```
make -f tensorflow/lite/micro/tools/make/Makefile \
TARGET="disco_f746ng" generate_micro_speech_mbed_project
```



SparkFun Edge is a small, low-power development board designed for edge AI and sensor-based projects. Image source: https://learn.sparkfun.com/

Mbed (now often referred to as Mbed OS) is an open-source real-time operating system (RTOS) for microcontroller units (MCUs) from Arm and its ecosystem partners. It provides a consistent, high-level abstraction layer, making it easier to write, build, and debug embedded applications across different hardware platforms.

Memory Management in TinyML

Static Memory Allocation Strategy:

- **No malloc()/free()** to avoid heap fragmentation
- **Pre-planned memory layout** for long-running applications
- **Fixed-size arena** passed at initialization
- **Immediate error** if arena too small

Memory Arena Concept:

```
// Application provides memory arena
constexpr int kTensorArenaSize = 60 * 1024;
uint8_t tensor_arena[kTensorArenaSize];

// Framework uses arena for all allocations
tflite::MicroInterpreter interpreter(model, resolver,
                                     tensor_arena, kTensorArenaSize);
```

uint8_t is a typedef in C and C++ that represents an unsigned 8-bit integer

Benefit: Predictable memory usage, no runtime allocation failures

Framework Selection Criteria

Three Key Evaluation Factors:

1. Model Requirements

- Supported operations and architectures
- Training vs inference capabilities *
- Quantization support needs *

2. Software Dependencies

- Operating system requirements
- Memory management capabilities
- Accelerator delegation support

3. Hardware Constraints

- Binary size limitations
- Memory footprint requirements
- Target processor architecture

Example: TensorFlow variants demonstrate these trade-offs

** Training is the process of adjusting a model's parameters using labeled data to learn patterns, while inference is using the trained model to make predictions on new, unseen data.*

** Quantization in machine learning is a technique to reduce the computational and memory requirements of neural networks by lowering the precision of the numbers used to represent model parameters and activations.*

TensorFlow Framework Comparison

Pattern: Progressive optimization for increasingly constrained environments

Aspect	TensorFlow	TensorFlow Lite	TF Lite Micro
Training	Yes	No	No
Inference	Yes (inefficient on edge)	Yes (efficient)	Yes (very efficient)
Operations	~1400	~130	~50
Binary Size *	3 MB+	100 KB	~10 KB
Memory *	~5 MB	300 KB	20 KB
OS Required	Yes	Yes	No
Accelerators	Yes	Yes	No

***Memory**: The runtime for inference on ultra-constrained devices (bare-metal or tiny RTOS) that can fit within tens of kilobytes to a few megabytes total.*
***Binary size**: The size of the compiled runtime library that you load onto a device. It's the portion of the software stack necessary to run the model and perform inference.*

Practical Example - Audio Processing

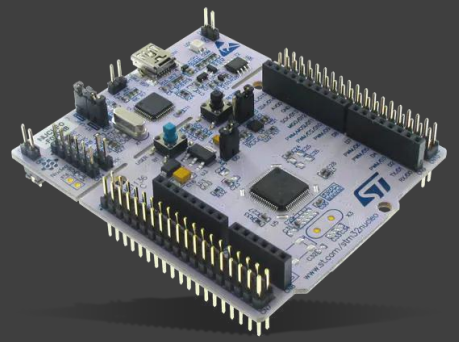
Problem: Cross-platform audio capture for wake word detection

Default Implementation (audio_provider.cc):

```
TfLiteStatus GetAudioSamples(tflite::ErrorReporter* error_reporter,
                             int start_ms, int duration_ms,
                             int* audio_samples_size,
                             int16_t** audio_samples) {
// Returns array of zeros - no real microphone
for (int i = 0; i < kMaxAudioSampleSize; ++i) {
        g_dummy_audio_data[i] = 0;
}
*audio_samples = g_dummy_audio_data;
return kTfLiteOk;
}
```

Specialized Implementation: Platform-specific versions in subfolders

- **disco_f746ng/audio_provider.cc** - STM32 with Mbed
- **osx/audio_provider.cc** - macOS development



STM32 is a family of microcontroller units (MCUs) and microprocessors from STMicroelectronics. it's used for Embedded devices, sensors, motor control, audio/video processing, IoT nodes, wearables, industrial automation, and more. Image source: https://www.mouser.hk/

Specialization Mechanism: Subfolder Pattern

Framework Structure:

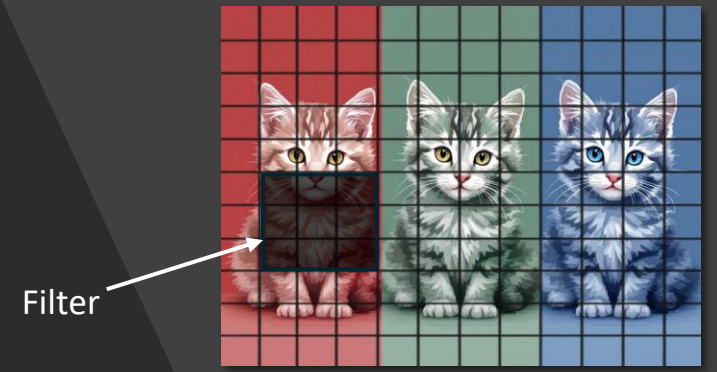
```
tensorflow/lite/micro/examples/micro_speech/
├── audio_provider.cc              # Default implementation
├── disco_f746ng/
│   └── audio_provider.cc          # STM32 specialized
version
├── osx/
│   └── audio_provider.cc          # macOS specialized
version
└── sparkfun_edge/
    └── audio_provider.cc          # SparkFun Edge version
```

Build System Logic:

- **Check TARGET parameter** (e.g., TARGET=disco_f746ng)
- **Search subfolders** for specialized implementations
- **Substitute specialized version** if found
- **Use default** if no specialization exists

Result: Same interface, platform-optimized implementation

CNNs refresher



What goes in

- Input: 3D tensor (H, W, C). Example: 224×224×3 (RGB); grayscale: C=1.

How it works

- Filters/Kernels: small matrices (e.g., 3×3) with depth C; each filter → 1 output channel.
- Convolution: slide, dot-product → feature score per location.
- Output feature maps: stack of channels (e.g., 64) showing where patterns appear.

Shapes

- One filter on H×W×C → H_out×W_out×1
- N filters → H_out×W_out×N
- H_out/W_out = floor((Input−Kernel)/Stride) + 1; use Padding “same” to keep size.

Stride & Padding

- Stride 1: near-same size; Stride 2: halves H,W.
- Padding: “valid” shrinks; “same” preserves.

BatchNorm + ReLU

- BN: normalize each channel; ReLU: max(0,x) adds non-linearity.

Fusion (Conv + BN + ReLU)

- Do together for speed and fewer memory reads.

Analogy

- Assembly line: pixels in → machines (filters) detect patterns → quality check (BN+ReLU).

Key takeaways

- Filters are the eyes; output channels are their answers; fusion makes it fast.

Performance Optimization Example

Depthwise convolution is a sliding-window (spatial) scan done one channel at a time: the kernel moves across height and width to detect local patterns, but each input channel gets its own filter instead of mixing channels. It's much cheaper, and we usually add a 1×1 pointwise layer afterward to mix channels.

- Used in mobile/embedded CNNs and tiny wake-word/speech models.
- Chosen for low compute, low memory, and lower power while keeping good accuracy.

Problem: Depthwise convolution bottleneck in speech model

Default Implementation:

```
// Nested loops - easy to understand, slow execution
for (int b = 0; b < batches; ++b) {
  for (int out_y = 0; out_y < output_height; ++out_y) {
    for (int out_x = 0; out_x < output_width; ++out_x) {
      for (int ic = 0; ic < input_depth; ++ic) {
        // ... computation
      }
    }
  }
}
```

Optimized Implementation (portable_optimized subfolder):

- **Precalculated array indices** (avoid repeated computation)
- **8-byte word loading** (multiple values per memory access)
- **Loop unrolling** for better instruction pipeline usage

Result: Significant speedup while maintaining correctness

Model File Format: FlatBuffers

FlatBuffers is a binary serialization format plus a schema compiler. You define a schema (what fields exist), then serialize data into a flat binary layout that can be read directly without parsing or allocating. Readers access fields via pointers/offsets—no unpacking step.

Why FlatBuffers?

- Zero-copy deserialization - direct memory access
- Same in-memory and serialized format
- Embedded directly in flash memory
- No parsing overhead

Schema Structure (schema.fbs):

```
table Model {
  version: uint;
  operator_codes: [OperatorCode];
  subgraphs: [SubGraph];
  buffers: [Buffer];
}

table SubGraph {
  tensors: [Tensor];
  operators: [Operator];
  inputs: [int];
  outputs: [int];
}

root_type Model;
```

Model Loading Example

Loading a Model:

```
// Map model data (no copying/parsing)
const tflite::Model* model =
    tflite::GetModel(g_tiny_conv_micro_features_model_data);

// Version check
if (model->version() != TFLITE_SCHEMA_VERSION) {
    error_reporter->Report("Schema version mismatch");
    return 1;
}

// Access subgraph
auto* subgraphs = model->subgraphs();
if (subgraphs->size() != 1) {
    error_reporter->Report("Only 1 subgraph supported");
    return 1;
}
subgraph_ = (*subgraphs)[0];
```

Key Point: Direct memory access, no data structure construction

Tensors and Buffers in FlatBuffers

Buffer: Raw data storage

```
table Buffer {
  data: [ubyte] (force_align: 16);  // 16-byte aligned raw data
}
```

Tensor: Metadata + buffer reference

```
table Tensor {
  shape: [int];                // Dimensions [batch, height,
width, channels]
  type: TensorType;            // float32, int8, etc.
  buffer: uint;                // Index into buffers array
  name: string;                // For debugging
  quantization: QuantizationParameters;  // For quantized
models
}
```

Separation: **Data** (buffers) vs **Metadata** (tensors)

Operator Graph Structure

Operator Definition:

```
table Operator {
  opcode_index: uint;          // Index into operator_codes
  inputs: [int];               // Input tensor indices
  outputs: [int];              // Output tensor indices
  builtin_options: BuiltinOptions; // Operation parameters
}
```

Execution Order:

- **Operators in topological order** - dependencies resolved
- **Sequential execution** from first to last operator
- **No graph analysis** needed at runtime

Example: Conv2D → ReLU → Dense

- **Conv2D** reads input tensors, writes to activation tensor
- **ReLU** reads activation tensor, writes to processed tensor
- **Dense** reads processed tensor, writes to output tensor

Hardware Integration Challenges

Supporting New Hardware Platforms:

Required: Debug logging capability

```
extern "C" void DebugLog(const char* s) {
// Platform-specific implementation
// Arduino: Serial.println(s)
// ARM: Semihosting SYS_WRITE0
// Mbed: pc.printf("%s", s)
}
```

Steps for New Platform:

1. **Implement DebugLog()** for your platform
2. **Create platform subfolder** (e.g., my_platform/)
3. **Add specialized implementations** as needed
4. **Generate project** with TARGET=my_platform

Result: All framework targets available on new platform

Cross-Platform Development Flow

Development Process:

```
1. Develop on x86 Linux/Mac
   ├── Fast compilation
   ├── Easy debugging
   └── Full toolchain

2. Test on target platform
   ├── Generate project for target
   ├── Build with target toolchain
   └── Flash and verify

3. Optimize for target
   ├── Add specialized implementations
   ├── Platform-specific optimizations
   └── Performance tuning
```

Tools Available:

- **Makefile builds** for automated testing
- **IDE project generation** for manual development
- **Continuous integration** for multiple platforms

The TinyML Advantage

Energy Efficiency:

- **< 1 mW power consumption**
- **Coin battery lasts 1 year**
- **Always-on sensing** capabilities

Coin cell batteries – Image source: Wikipedia

Applications Enabled:

- **Environmental monitoring** without infrastructure
- **Predictive maintenance** in remote locations
- **Wildlife tracking** with minimal disturbance
- **Health monitoring** with wearable devices

Key Insight: Local processing eliminates energy costs of data transmission

Trade-off: Model complexity vs deployment ubiquity

Week 1: Key Takeaways

Framework Evolution:

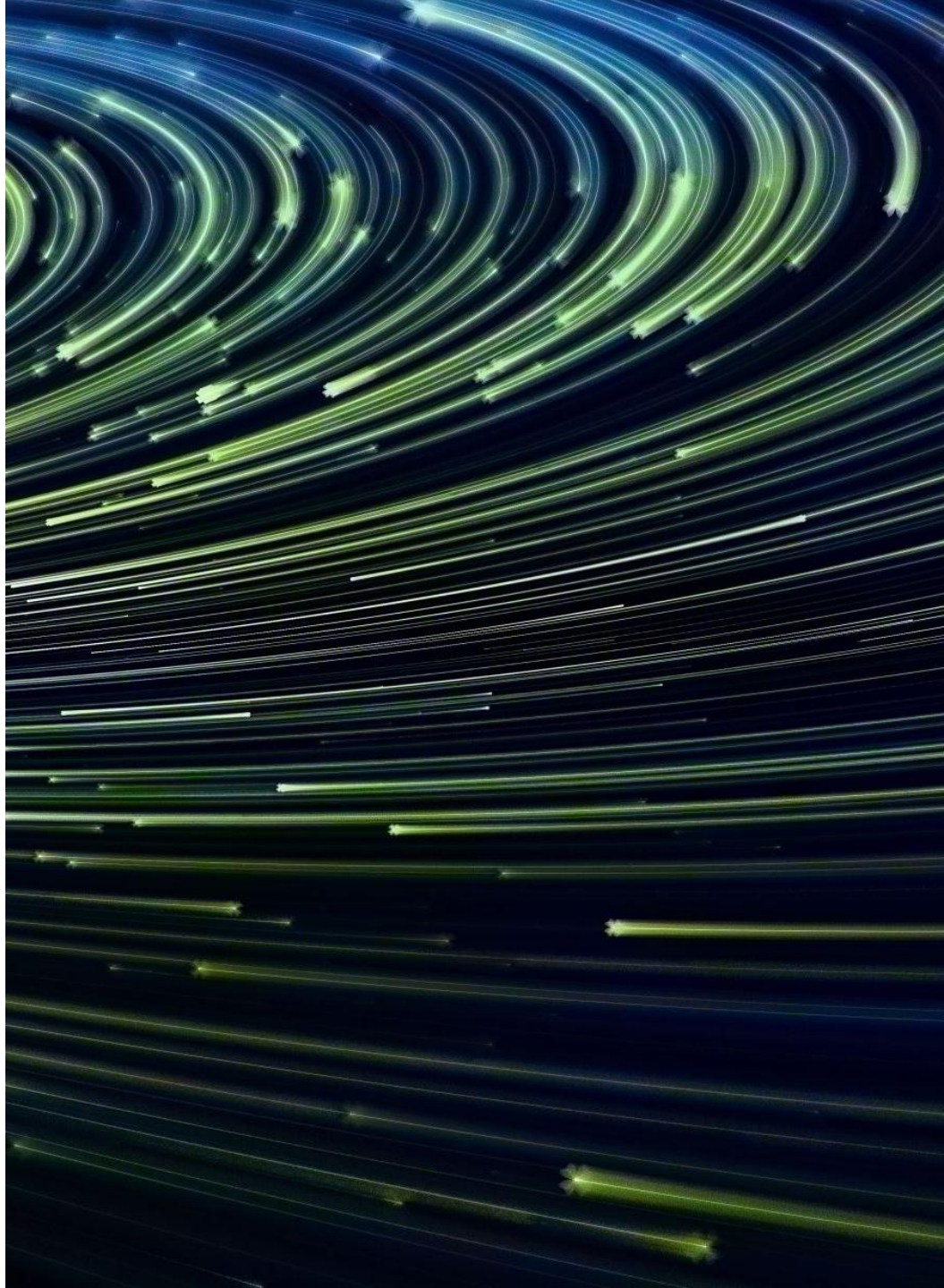
- **Progressive abstraction** from BLAS/LAPACK to modern frameworks
- **Each generation** addressed new computational challenges
- **Hardware advances** drove framework architectural changes

Framework Fundamentals:

- **Computational graphs** enable automatic differentiation and optimization
- **Tensor data structures** unify diverse ML computations
- **Programming models** balance flexibility vs optimization
- **Execution models** trade development ease for runtime performance

Specialization Necessity:

- **Different environments** require different optimizations
- **Cloud, edge, mobile, TinyML** each have unique constraints
- **Framework selection** depends on specific requirements



Looking Ahead: Week 2 - Automatic Differentiation

Next Week's Focus:

- **Deep dive** into automatic differentiation mechanisms
- **Forward vs reverse mode** detailed analysis
- **Memory and compute constraints** in embedded systems
- **Training vs inference** considerations across platforms

Required Readings for Week 2:

- **Book 1 - Volume 1:** Neural Computation
- **Book 1 - Volume 1:** ML Frameworks
- **Book 1 - Volume 1:** Model Training
- **Book 2:** Chapter 3: Getting Up to Speed on Machine Learning

Preparation: Review calculus chain rule and matrix derivatives

Assignment — Cross-Platform Framework Implementation (Due: Week 3 – Sep 15 – 22:59)

Objective

•Implement MNIST CNN in TensorFlow → convert to TFLite → optionally deploy with TFLite Micro. Compare size, accuracy, memory, speed.

•Important note

•Hardware NOT required. You may run everything in simulation on your laptop. If you have an Arduino/MCU and want to try it, great—but optional.

Parts & Points

•Part 1 (25): TensorFlow CNN (Conv 8×3×3 + ReLU → MaxPool 2×2 → Flatten → Dense 16 + ReLU → Dense 10). Train 5 epochs; target >95% test acc; save mnist_cnn_model.keras.

•Part 2 (25): TFLite conversion (+ quantized variant). Produce .tflite files; report size and accuracy differences.

•Part 3 (25): TFLite Micro deployment

- •Preferred path: simulate on desktop with TFLM (no MCU needed) and report arena size.
- •Optional: flash to MCU (e.g., Arduino) using xxd -i and run a test image.

•Part 4 (25): 2–3 pages analysis: size, accuracy, RAM/flash, inference time/energy (simulated OK), trade-offs, quantization impact.

Deliverables

•Code: part1_tensorflow.py, part2_tflite_conversion.py, part3_micro_deployment.cc, model_data.cc, Makefile/build steps.

•Models: mnist_cnn_model.keras, mnist_model.tflite, mnist_model_quantized.tflite.

•Docs: performance_analysis.pdf, README.md, sample data/screenshots/logs.

Software to install

•Python 3.9+, pip; TensorFlow 2.x; numpy; matplotlib

•TFLite APIs (via TensorFlow) or tflite-runtime (optional)

•C/C++ toolchain (gcc/clang or ARM GCC), Make/CMake, xxd

•Optional for real boards: Arduino IDE/ESP-IDF, CMSIS-NN

Final Project Proposal — Federated Edge Intelligence (Due: Week 8 – Oct 27, 2026 – 23:59)

Length: 3–4 pages PDF (exclude references)
Filename: StudentId_LastName_FirstName_ProjectProposal.pdf

Important note

Real hardware NOT required. A comprehensive simulation is fully acceptable. If you have devices (e.g., Arduino/edge boards), you may use them—but optional.

Required sections

1.Problem & Motivation (0.5–1 p): real-world need, why FL fits, impact, participants + constraints.
2.System Architecture (1–1.5 p): diagram, topology (centralized/decentralized/hierarchical), device heterogeneity, comms/dataflow, privacy/security choices.
3.Technical Approach (1–1.5 p): FL algorithm + aggregation, privacy tech (DP, secure aggregation), hardware-aware optimizations, compression/deployment, 2–3 advanced features.
4.Implementation Plan (0.5–1 p): simulation or hardware path, languages/frameworks/tools, timeline/milestones, risks + mitigations, dev/test environment.
5.Evaluation (0.5 p): success metrics, baselines, privacy/security tests, scalability/efficiency plan.

Project options

1: Federated Wildlife Conservation
2: Federated Smart Agriculture
3: Federated Personalized Education
4: Federated Home Energy Optimization
5: Your own idea (requires approval; equivalent complexity)

Software to install (for simulation)

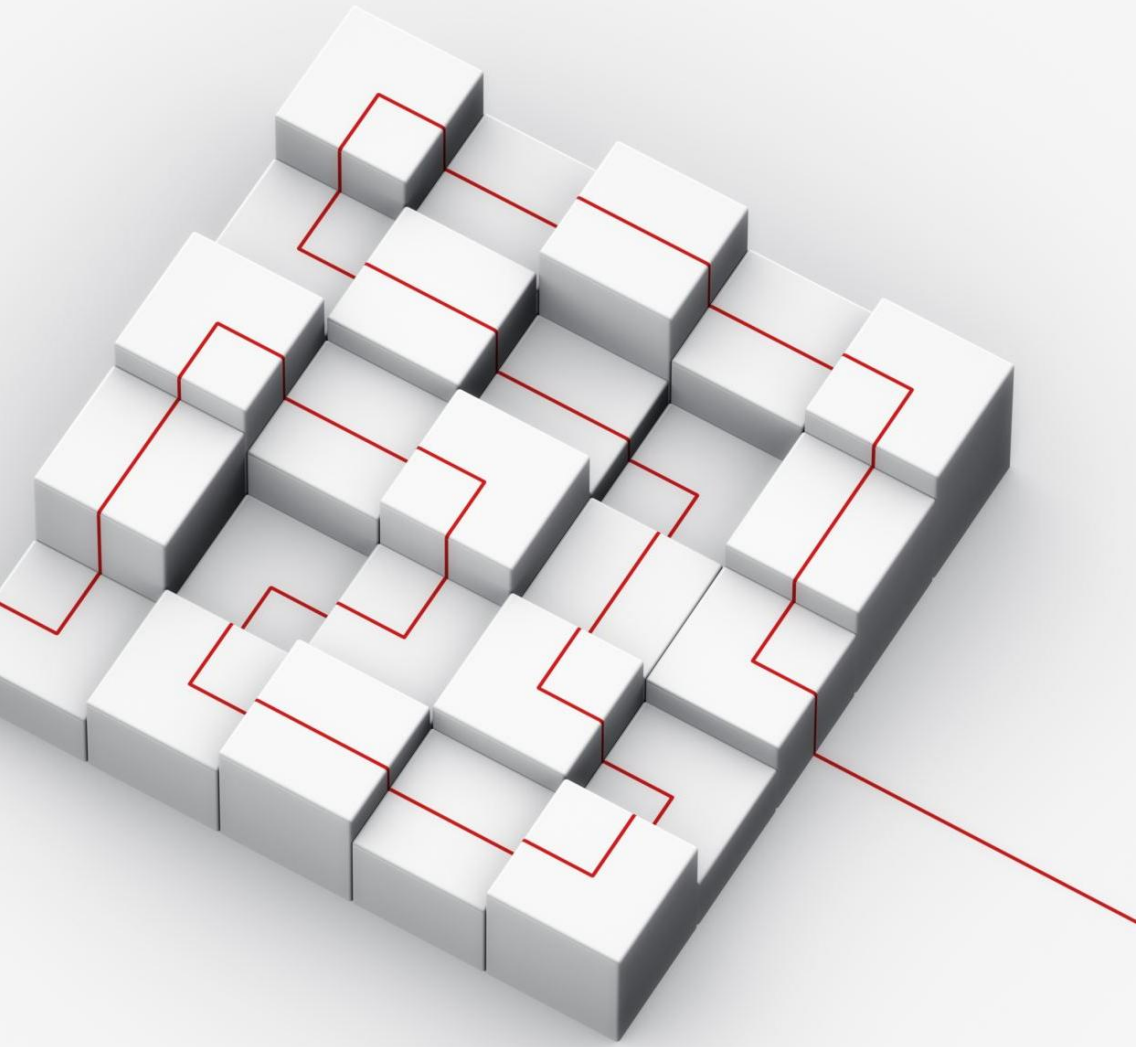
•Python 3.9+, PyTorch or TensorFlow, Flower/FedML/FedAvg refs, NumPy/Pandas
•Optional: Docker, CUDA toolkit (if using GPUs), graph tools (NetworkX), Jupyter
•Optional hardware stack: Arduino/edge SDKs if you choose devices

Evaluation criteria (weights)

•Architecture 25% | Technical 35% | Implementation 25% | Presentation 15%

For this proposal, the project is individual by default, but teams of up to 4 are acceptable with instructor approval.

If you choose a team:
•Submit one proposal with all member names and student IDs.
•Add a “Team Plan & Roles” subsection outlining contributions, division of work, and coordination.
•Scope should scale with team size (e.g., larger architecture, richer evaluation, or additional advanced features).
•Clearly attribute any external resources or collaborations.



Questions and Discussion

Today's Topics:

- **Framework evolution** and design principles
- **Static vs dynamic** computational graphs
- **TinyML specialization** and constraints
- **Cross-platform deployment** strategies

Think About:

- Which framework approach would you choose for your application?
- How do hardware constraints influence framework design?
- What trade-offs are you willing to make for deployment efficiency?